

Resumo de Estudo — Equivalência AP ↔ GLC (Parte 2)

Conversão de AP para GLC • Seção 2.2

Na aula anterior provamos que toda GLC tem um AP equivalente. Agora fechamos a equivalência com a **direção oposta**: dado qualquer autômato com pilha, construir uma GLC que gera exatamente a mesma linguagem. Esta é a parte mais abstrata, mas a ideia central é elegante.

Lema 2.27

Se um autômato com pilha reconhece alguma linguagem, então essa linguagem é livre-do-contexto. Juntando com o Lema 2.21 (aula anterior), concluímos: uma linguagem é livre-do-contexto **se e somente se** algum AP a reconhece.

1. Primeiro, padronizamos o AP

Para facilitar a construção, assumimos que o AP tem três propriedades. Qualquer AP pode ser facilmente adaptado para satisfazê-las:

Três requisitos sobre o AP

- 1. Tem um **único estado de aceitação**, q_{aceita} .
- 2. **Esvazia a pilha antes de aceitar** (a pilha está vazia ao aceitar).
- 3. Cada transição ou **empilha** um símbolo, ou **desempilha** um símbolo — **nunca as duas coisas ao mesmo tempo**, e nunca nenhuma das duas.

Como garantir o requisito 3 (o mais técnico)

Se alguma transição faz empilhamento e desempilhamento simultâneos, nós a "desdobramos" em duas, passando por um estado auxiliar:

```
Transição    p_i --(a, b → c)--> p_j
vira duas:   p_i --(a, b → λ)--> p_aux --(λ, λ → c)-->
p_j
```

A primeira só desempilha (b), a segunda só empilha (c). Transições que não mexem na pilha também são ajustadas de forma análoga (empilha-desempilha um símbolo "fictício").

2. A ideia central: variáveis A_{pq}

Queremos uma GLC G tal que G gere uma cadeia x **exatamente quando** x leva o AP do estado inicial ao estado de aceitação. A grande ideia é criar uma variável para **cada par de estados**:

Definição das variáveis

Para cada par de estados (p, q) , criamos uma variável A_{pq} . Ela deve gerar **todas as cadeias x que levam o AP do estado p (com a pilha vazia) ao estado q (com a pilha vazia)**.

A variável inicial da gramática é $A_{(q_0, q_{aceita})}$ — as cadeias que levam do estado inicial ao de aceitação, começando e terminando com a pilha vazia. É exatamente a linguagem do AP.

Por que "pilha vazia no início e no fim"? Graças ao requisito 2, o AP esvazia a pilha antes de aceitar; e ele começa com a pilha vazia. Então a computação inteira (do inicial ao de aceitação) é justamente um trajeto "de pilha vazia a pilha vazia" — o que a variável inicial $A_{(q_0, q_{aceita})}$ descreve.

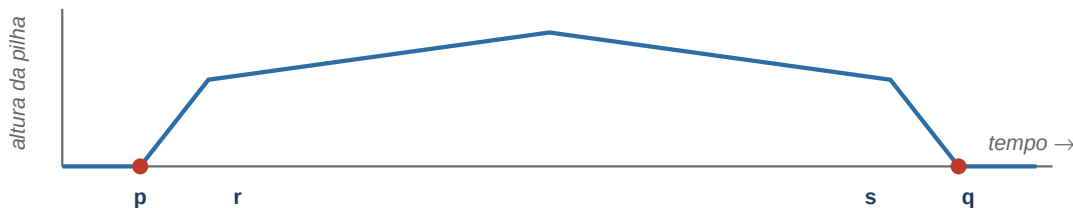
3. As duas situações de um trajeto $p \rightarrow q$

Considere um trajeto do AP de p a q , começando e terminando com a pilha vazia, lendo a cadeia x . Pense em como a **altura da pilha** evolui ao longo do tempo. Como o AP sai do vazio e volta ao vazio, há exatamente **duas situações possíveis**:

Caso 1 — a pilha só volta a ficar vazia no final

Durante todo o trajeto (exceto no início e no fim) a pilha permanece não-vazia. Isso significa que o **primeiro símbolo empilhado só é desempilhado no último passo**.

Caso 1: pilha vazia só nas pontas



O primeiro movimento (lendo «a») empilha um símbolo t e vai de p para r ; o último movimento (lendo «b») desempilha esse mesmo t e vai de s para q . No meio, o AP vai de r a s sem nunca esvaziar a pilha. Isso dá a regra:

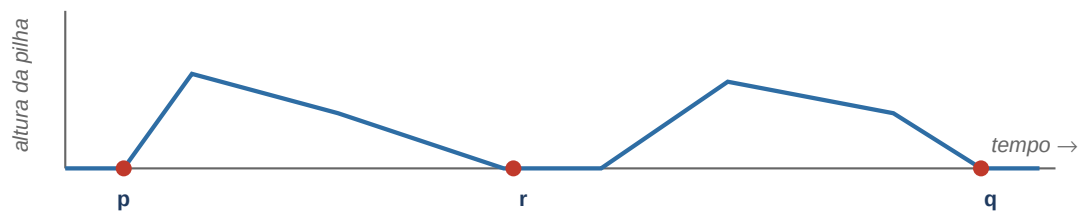
$$A_{pq} \rightarrow a A_{rs} b$$

onde $x = a \cdot y \cdot b$, e y é a subcadeia que leva de r a s (num nível de pilha acima do t). Formalmente: se $\delta(p, a, \epsilon)$ contém (r, t) e $\delta(s, b, t)$ contém (q, ϵ) , então $A_{pq} \rightarrow a A_{rs} b$ está em G .

Caso 2 — a pilha fica vazia em algum ponto do meio

Existe um estado intermediário r onde a pilha volta a ficar vazia antes de chegar em q . Nesse caso, podemos **quebrar o trajeto em dois** pedaços independentes: de p a r , e de r a q , ambos "de pilha vazia a pilha vazia".

Caso 2: pilha esvazia num ponto intermediário r



A cadeia se divide como $x = y \cdot z$, onde y leva de p a r e z leva de r a q. Isso dá a regra:

$$A_{pq} \rightarrow A_{pr} A_{rq}$$

4. A construção formal da gramática

Reunindo os dois casos, mais o caso base, definimos as regras de G . Seja $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{aceita}\})$. As variáveis de G são $\{A_{pq} \mid p, q \in Q\}$, e a variável inicial é $A_{(q_0, q_{aceita})}$.

As regras de G

- **Caso 1:** para cada $p, q, r, s \in Q$, cada $t \in \Gamma$ e cada $a, b \in \Sigma_\epsilon$, se $\delta(p, a, \epsilon)$ contém (r, t) e $\delta(s, b, t)$ contém (q, ϵ) , então adicione a regra $A_{pq} \rightarrow a A_{rs} b$.
- **Caso 2:** para cada $p, q, r \in Q$, adicione a regra $A_{pq} \rightarrow A_{pr} A_{rq}$.
- **Caso base:** para cada $p \in Q$, adicione a regra $A_{pp} \rightarrow \epsilon$ (a cadeia vazia leva um estado a si mesmo com a pilha vazia).

Intuição das regras

- A regra do Caso 1 ($A_{pq} \rightarrow a A_{rs} b$) reconhece que os símbolos das pontas «a» e «b» estão "casados" pela pilha — o t empilhado no início é o t desempilhado no fim, como um par de parênteses correspondentes.
- A regra do Caso 2 ($A_{pq} \rightarrow A_{pr} A_{rq}$) é uma simples concatenação de dois trajetos independentes emendados no estado r .
- A regra base ($A_{pp} \rightarrow \epsilon$) encerra a recursão: um trajeto de tamanho zero.

5. Por que a construção funciona (esboço da prova)

É preciso provar a afirmação-chave nas duas direções (é um "se e somente se"):

Afirmação 2.31

A_{pq} gera a cadeia x se e somente se x leva o AP de p (pilha vazia) a q (pilha vazia).

Direção «gera $\Rightarrow$ leva»

Por indução no número de passos da derivação de $A_{pq} \Rightarrow^* x$. No caso base, a derivação tem um passo ($A_{pp} \rightarrow \epsilon$), e de fato a cadeia vazia leva p a p sem mexer na pilha. No passo indutivo, a primeira regra usada é do Caso 1 ou do Caso 2, e a hipótese de indução se aplica às subderivações (A_{rs} ou A_{pr}, A_{rq}), reconstruindo o trajeto no AP.

Direção «leva $\Rightarrow$ gera»

Por indução no número de passos da computação do AP. Assuma verdadeiro para computações de comprimento $\leq k$ e prove para $k + 1$. Se x leva p a q em $k+1$ passos (pilhas vazias nas pontas), analisamos os dois casos pela forma como a pilha se comporta:

- **Caso 1** (pilha vazia só nas pontas): o primeiro símbolo empilhado t é o último a sair. Sejam r o 2º estado e s o penúltimo, e a, b os símbolos lidos no 1º e último movimentos. Então $\delta(p, a, \epsilon) \ni (r, t)$ e $\delta(s, b, t) \ni (q, \epsilon)$, logo $A_{pq} \rightarrow a A_{rs} b$ está em G . A subcadeia y (com $x = a \cdot y \cdot b$) leva r a s em $k-1$ passos sem tocar em t ; pela hipótese de indução, $A_{rs} \Rightarrow^* y$. Logo $A_{pq} \Rightarrow^* x$.
- **Caso 2** (pilha esvazia num ponto intermediário r): cada trecho (de p a r e de r a q) tem no máximo k passos, e $x = y \cdot z$. Pela hipótese de indução, $A_{pr} \Rightarrow^* y$ e $A_{rq} \Rightarrow^* z$. Como $A_{pq} \rightarrow A_{pr} A_{rq}$ está em G , temos $A_{pq} \Rightarrow^* x$.

Provadas as duas direções, a variável inicial $A(q_0, q_{aceita})$ gera exatamente as cadeias aceitas pelo AP — ou seja, $L(G) = L(P)$. ■

Conclusão dos dois lemas — o teorema completo

Lema 2.21 (Parte 1): toda GLC tem um AP equivalente. Lema 2.27 (Parte 2): todo AP tem uma GLC equivalente. Juntos, eles estabelecem que **GLCs e APs descrevem exatamente a mesma classe de linguagens — as linguagens livres-do-contexto**. Podemos, portanto, escolher a ferramenta mais conveniente (gerar com GLC ou reconhecer com AP) para cada problema.

Exercícios resolvidos

Exercício 1 — Quais são as três formas de regra na construção $AP \rightarrow GLC$?

São exatamente três tipos:

- $A_{pq} \rightarrow a A_{rs} b$ (Caso 1): quando um símbolo empilhado no primeiro movimento é desempilhado só no último — os terminais das pontas ficam "casados".
- $A_{pq} \rightarrow A_{pr} A_{rq}$ (Caso 2): quando a pilha esvazia num estado intermediário r , dividindo o trajeto em dois.
- $A_{pp} \rightarrow \varepsilon$ (caso base): trajeto vazio de um estado para si mesmo.

A variável inicial é $A_{(q_0, q_{aceita})}$. ✓

Exercício 2 — Por que exigimos que o AP esvazie a pilha antes de aceitar?

Porque a variável inicial $A_{(q_0, q_{aceita})}$ foi definida para gerar as cadeias que levam do estado inicial ao de aceitação **com a pilha vazia nas duas pontas**. Se o AP pudesse aceitar com a pilha não-vazia, essas computações não seriam capturadas por $A_{(q_0, q_{aceita})}$, e a gramática geraria a linguagem errada. Forçar o esvaziamento (requisito 2) faz a definição de A_{pq} "de pilha vazia a pilha vazia" coincidir exatamente com a condição de aceitação. ✓

Exercício 3 — Por que separar empilhar de desempilhar (requisito 3)?

A construção das regras se apoia em identificar, em cada movimento, se ele **empilha** ou **desempilha** um símbolo — isso é o que permite reconhecer o "casamento" do Caso 1 (um símbolo empilhado no início e retirado no fim) e o esvaziamento do Caso 2. Se uma transição fizesse as duas operações ao mesmo tempo, não seria possível rastrear de forma limpa a evolução da altura da pilha. Por isso desdobramos cada transição mista em duas (uma só de desempilhar, outra só de empilhar), passando por um estado auxiliar. ✓

Exercício 4 — Interprete a regra $A_{pq} \rightarrow a A_{rs} b$ como "parênteses"

Pense no símbolo empilhado t como um par de parênteses: « a » abre (empilha t) e « b » fecha (desempilha t). Tudo o que acontece entre eles — a subcadeia y gerada por A_{rs} — ocorre "dentro" desse par, num nível de pilha acima de t . É a mesma estrutura recursiva que vimos em gramáticas como $S \rightarrow aSb$ (a linguagem $a^n b^n$) ou nas expressões parentizadas: um delimitador de abertura casado com um de fechamento, com conteúdo aninhado no meio. Essa correspondência é a razão de o AP e a GLC descreverem a mesma linguagem. ✓